Introduction

The pymssql package consists of two extension modules:

- **pymssql**. This is the DB-API compliant module. Use it if your primary goal is to follow DB-API standard.
- **_mssql**. This is lower level module that talks directly to SQL Server. It has many useful features. You can use it if you don't need DB-API compliance. It's pretty straighforward to use, as you will find below.

pymssql examples (strict DB-API compliance):

```
import pymssql
conn = pymssql.connect(host='SQL01', user='user', password='password', database='mydatabase')
cur = conn.cursor()
cur.execute('CREATE TABLE persons(id INT, name VARCHAR(100))')
cur.executemany("INSERT INTO persons VALUES(%d, %s)", \
   [ (1, 'John Doe'), (2, 'Jane Doe') ])
conn.commit()  # you must call commit() to persist your data if you don't set autocommit to True

cur.execute('SELECT * FROM persons WHERE salesrep=%s', 'John Doe')
row = cur.fetchone()
while row:
   print "ID=%d, Name=%s" % (row[0], row[1])
   row = cur.fetchone()

# if you call execute() with one argument, you can use % sign as usual
# (it loses its special meaning).
cur.execute("SELECT * FROM persons WHERE salesrep LIKE 'J%'")

conn.close()
```

You can also use iterators instead of `while` loop. Iterators are DB-API extensions, and are available since pymssql 1.0.

Rows as dictionaries

Since pymssql 1.0.2 rows can be fetched as dictionaries instead of tuples. This allows for accessing columns by name instead of index.

```
import pymssql
conn = pymssql.connect(host='SQL01', user='user', password='password', database='mydatabase',as
_dict=True)
cur = conn.cursor()

cur.execute('SELECT * FROM persons WHERE salesrep=%s', 'John Doe')
for row in cur:
   print "ID=%d, Name=%s" % (row['id'], row['name'])

conn.close()
```

_mssql examples (no DB-API overhead)

This module allows for easy communication with SQL Server.

Quickstart usage of various features:

```
import _mssql
conn = _mssql.connect(server='SQL01', user='user', password='password', \
   database='mydatabase')
conn.execute_non_query('CREATE TABLE persons(id INT, name VARCHAR(100))')
conn.execute_non_query("INSERT INTO persons VALUES(1, 'John Doe')")
conn.execute_non_query("INSERT INTO persons VALUES(2, 'Jane Doe')")
```

```
# how to fetch rows from a table
conn.execute_query('SELECT * FROM persons WHERE salesrep=%s', 'John Doe')
for row in conn:
   print "ID=%d, Name=%s" % (row['id'], row['name'])
```

```
# examples of other query functions
numemployees = conn.execute_scalar("SELECT COUNT(*) FROM employees")
numemployees = conn.execute_scalar("SELECT COUNT(*) FROM employees WHERE name
LIKE 'J%'")    # note that '%' is not a special character here
employeedata = conn.execute_row("SELECT * FROM employees WHERE id=%d", 13)
```

```
# how to fetch rows from a stored procedure
conn.execute_query('sp_spaceused')   # sp_spaceused without arguments returns 2 result sets
res1 = [ row for row in conn ]      # 1st result
res2 = [ row for row in conn ]      # 2nd result
```

```
# how to get an output parameter from a stored procedure
sqlcmd = """
DECLARE @res INT
EXEC usp_mystoredproc @res OUT
SELECT @res
"""
res = conn.execute_scalar(sqlcmd)
```

```
# how to get more output parameters from a stored procedure
sqlcmd = """
DECLARE @res1 INT, @res2 TEXT, @res3 DATETIME
```

```
EXEC usp_getEmpData %d, %s, @res1 OUT, @res2 OUT, @res3 OUT
SELECT @res1, @res2, @res3
"""
res = conn.execute_row(sqlcmd, (13, 'John Doe'))
```

```
# examples of queries with parameters
conn.execute_query('SELECT * FROM empl WHERE id=%d', 13)
conn.execute_query('SELECT * FROM empl WHERE name=%s', 'John Doe')
conn.execute_query('SELECT * FROM empl WHERE id IN (%s)', ((5, 6),))
conn.execute_query('SELECT * FROM empl WHERE name LIKE %s', 'J%')
conn.execute_query('SELECT * FROM empl WHERE name=%(name)s AND city=%(city)s', \
   { 'name': 'John Doe', 'city': 'Nowhere' } )
conn.execute_query('SELECT * FROM cust WHERE salesrep=%s AND id IN (%s)', \
   ('John Doe', (1, 2, 3)))
conn.execute_query('SELECT * FROM empl WHERE id IN (%s)', (tuple(xrange(4)),))
conn.execute_query('SELECT * FROM empl WHERE id IN (%s)', \
   (tuple([3, 5, 7, 11]),))
```

```
conn.close()
```

Please note the usage of iterators and ability to access results by column name. Also please note that parameters to `connect` method have different names than in pymssql module.

An example of exception handling:

```
import _mssql
try:
   conn = _mssql.connect(server='SQL01', user='user', password='password', \
      database='mydatabase')
   conn.execute_non_query('CREATE TABLE t1(id INT, name VARCHAR(50))')
except _mssql.MssqlDatabaseException,e:
   if e.number == 2714 and e.severity == 16:
      # table already existed, so quieten the error
   else:
      raise # re-raise real error
finally:
   conn.close()
```

Please see more info on exceptions below.

Architecture and configuration

pymssql on Windows

pymssql on Windows doesn't require any additional components to be installed. The only required library, ntwdblib.dll, is included with pymssql, and it's all that is needed to make connections to Microsoft SQL Servers. It is called DB Libary for C, and is documented here. This library is responsible for low level communication with SQL Servers, and is extensively used by _mssql module.

Typically you don't need any configuration in order to make it work. However I wrote a few paragraphs about more sophisticated configuration in Advanced information.

On Windows in addition to authenticating to SQL using user name and password, you can also authenticate using so called Windows Authentication, or Trusted Connection. In this mode SQL Server validates the connection using user's *security token* provided by the operating system. In other words, user's identity is confirmed by Windows, so SQL trusts it (hence trusted connection). It is suggested method of connecting, because is eliminates the need for hard coding passwords in clear text in scripts.

```
# An example on how to connect using Windows Integrated Authentication.
conn = _mssql.connect('sqlhost', trusted=True)
conn = pymssql.connect(host='sqlhost', trusted=True)
```

pymssql on Linux/*nix

Linux/*nix on this webpage refers to any operating system different than Microsoft Windows, i.e. Linux, BSD, Solaris, MacOS etc.
pymssql on these platforms require a low level driver that is able to speak to SQL Servers. This driver is implemented by FreeTDS package. FreeTDS can speak to both Microsoft SQL Servers and Sybase servers, however there is a problem with dates, which is described in more details on FreeTDS and Dates page.

You will need to configure FreeTDS. It needs nonempty configuration in order to work. Below you will find a simple examples, but for more sophisticated configurations you will need to consult FreeTDS User Guide.

```
# Minimal valid freetds.conf that is known to work.
[global]
    tds version = 7.0
```

You can connect using explicit host name and port number:

```
conn = _mssql.connect('sqlhost:portnum', 'user', 'password')
conn = pymssql.connect(host='sqlhost:portnum', user='user', password='password')
```

sSample FreeTDS configuration:

```
# The default instance on a host.
[SQL_A]
    host = sqlserver.example.com
    port = 1433
    tds version = 7.0

# Second (named) instance on the same host. The instance name is
# not used on Linux/*nix.
```

```
# This instance is configured to listen on TCP port 1444:
#  - for SQL 2000 use Enterprise Manager, Network configuration, TCP/IP,
#    Properties, Default port
#  - for SQL 2005 and newer use SQL Server Configuration Manager,
#    Network Configuration, Protocols for ..., TCP/IP, IP Addresses tab,
#    typically IPAll at the end.
[SQL_B]
    host = sqlserver.example.com
    port = 1444
    tds version = 7.0
```

An example of how to use the above configuration in _mssql:

```
conn_a = _mssql.connect('SQL_A', 'userA', 'passwordA')
conn_b = _mssql.connect('SQL_B', 'userB', 'passwordB')
```

pymssql module reference

pymssqlCnx class

This class represents an MS SQL database connection. You can create an instance of this class by calling constructor pymssql.connect(). It accepts following arguments. Note that in most cases you will want to use keyword arguments, instead of positional arguments.

dsn
colon-delimited string of the form host:dbase:user:pass:opt:tty, primarily for compatibility with previous versions of pymssql.
user
database user to connect as.
password
user's password.
trusted
bolean value signalling whether to use Windows Integrated Authentication to connect instead of SQL autentication with user and password [*Windows only*]
host
database host and instance you want to connect to. Valid examples are:

r'.\SQLEXPRESS' -- SQLEXPRESS instance on local machine [*Windows only*]
r'(local)\SQLEXPRESS' -- same as above [*Windows only*]
r'SQLHOST' -- default instance at default port [*Windows only*]
r'SQLHOST' -- specific instance at specific port set up in freetds.conf [*Linux/*nix only*]
r'SQLHOST,1433' -- specified TCP port at specified host
r'SQLHOST:1433' -- the same as above
r'SQLHOST,5000' -- if you have set up an instance to listen on port 5000
r'SQLHOST:5000' -- the same as above

'.' (the local host) is assumed if host is not provided.

database
the database you want initially to connect to, by default SQL Server selects the database which is set as default for specific user.
timeout
query timeout in seconds, default is 0 (wait indefinitely).
login_timeout
timeout for connection and login in seconds, default 60.
charset
character set with which to connect to the database.
as_dict
whether rows should be returned as dictionaries instead of tuples *(added in pymssql 1.0.2)*.
max_conn
how many simultaneous connections to allow; default is 25, maximum on Windows is 1474559; trying to set it to higher value results in error 'Attempt to set maximum number of DBPROCESSes lower than 1.' (error 10073 severity 7) *(added in pymssql 1.0.2)*.

pymssqlCnx object properties.

This class has no useful properties and data members.

pymssqlCnx object methods.

autocommit(status)

status is a boolean value. This method turns autocommit mode on or off. By default, autocommit mode is off, what means every transaction must be explicitly committed if changed data is to be persisted in the database. You can turn autocommit mode on, what means every single operation commits itself as soon as it succeeds.

close()
Close the connection.

cursor()
Return a cursor object, that can be used to make queries and fetch results from the database.

commit()
Commit current transaction. **You must call this method to persist your data** if you leave autocommit at its default value, which isFalse. See also pymssql example at the top of this page.

rollback()
Roll back current transaction.

pymssqlCursor class

This class represents a Cursor (in terms of Python DB-API specs) that is used to make queries against the database and obtaining results. You create pymssqlCursor instances by calling cursor() method on an open pymssqlCnx connection object.

pymssqlCursor object properties.

rowcount
Returns number of rows affected by last operation. In case of SELECT statements it returns meaningful information only after all rows have been fetched.

connection
This is the extension of the DB-API specification. Returns a reference to the connection object on which the cursor was created.

lastrowid
This is the extension of the DB-API specification. Returns identity value of last inserted row. If previous operation did not involve inserting a row into a table with identity column, None is returned.

rownumber
This is the extension of the DB-API specification. Returns current 0-based index of the cursor in the result set.

pymssqlCursor object methods.

close()
Close the cursor. The cursor is unusable from this point.

execute(operation)
execute(operation, params)
operation is a string and params, if specified, is a simple value, a tuple, or None.
Performs the operation against the database, possibly replacing parameter placeholders with provided values. This should be preferred method of creating SQL commands, instead of concatenating strings manually, what makes a potential of SQL Injection attacks. This method accepts the same formatting as Python's builtin string interpolation operator.
If you call execute() with one argument, you can use % sign as usual in your query

string, for example in LIKE operator (it loses its special meaning). See the example at the top.

executemany(operation, params_seq)
operation is a string and params_seq is a sequence of tuples (e.g. a list). Execute a database operation repeatedly for each element in parameter sequence.

fetchone()
Fetch the next row of a query result, returning a tuple, or a dictionary if as_dict was passed to pymssql.connect(), or None if no more data is available.
Raises OperationalError if previous call to execute*() did not produce any result set or no call was issued yet.

fetchmany(size=None)
Fetch the next batch of rows of a query result, returning a list of tuples, or a list of dictionaries if as_dict was passed topymssql.connect(), or an empty list if no more data is available. You can adjust the batch size using the size parameter, which is preserved across many calls to this method. Raises OperationalError if previous call to execute*() did not produce any result set or no call was issued yet.

fetchall()
Fetch all remaining rows of a query result, returning a list of tuples, or a list of dictionaries if as_dict was passed topymssql.connect(), or an empty list if no more data is available. Raises OperationalError if previous call to execute*() did not produce any result set or no call was issued yet.

fetchone_asdict()
Warning: this method is not part of DB-API. This method is deprecated as of pymsssql 1.0.2. It was replaced by as_dict parameter topymssql.connect()
Fetch the next row of a query result, returning a dictionary, or None if no more data is available. Data can be accessed by 0-based numeric column index, or by column name. Raises OperationalError if previous call to execute*() did not produce any result set or no call was issued yet.

fetchmany_asdict(size=None)
Warning: this method is not part of DB-API. This method is deprecated as of pymsssql 1.0.2. It was replaced by as_dict parameter topymssql.connect()
Fetch the next batch of rows of a query result, returning a list of dictionaries. An empty list is returned if no more data is available. Data can be accessed by 0-based numeric column index, or by column name. You can adjust the batch size using the size parameter, which is preserved across many calls to this method.
Raises OperationalError if previous call to execute*() did not produce any result set or no call was issued yet.

fetchall_asdict()
Warning: this method is not part of DB-API. This method is deprecated as of pymsssql 1.0.2. It was replaced by as_dict parameter topymssql.connect()
Fetch all remaining rows of a query result, returning a list of dictionaries. An empty list is returned if no more data is available. Data can be accessed by 0-based numeric column index, or by column name. Raises OperationalError if previous call to execute*() did not produce any result set or no call was issued yet.
The idea and original implementation of this method by Sterling Michel <sterlingmichel_at_gmail_dot_com>

nextset()
This method makes the cursor skip to the next available result set, discarding any remaining rows from the current set. Returns True value if next result is available, None if not.

__iter__(), next()
These methods faciliate Python iterator protocol. You most likely will not call them directly, but indirectly by using iterators.

setinputsizes(), setoutputsize()
These methods do nothing, as permitted by DB-API specs.

_mssql module reference

_mssql module properties

login_timeout
Timeout for connection and login in seconds, default 60.

min_error_severity
Minimum severity of errors at which to begin raising exceptions. The default value of 6 should be appropriate in most cases.

MssqlConnection class

This class represents an MS SQL database connection. You can make queries and obtain results through a database connection.

You can create an instance of this class by calling constructor _mssql.connect(). It accepts following arguments. Note that you can use keyword arguments, instead of positional arguments.

server
database server and instance you want to connect to. Valid examples are:

r'.\SQLEXPRESS' -- SQLEXPRESS instance on local machine [*Windows only*]
r'(local)\SQLEXPRESS' -- same as above [*Windows only*]
r'SQLHOST' -- default instance at default port [*Windows only*]
r'SQLHOST' -- specific instance at specific port set up in freetds.conf [*Linux/*nix only*]
r'SQLHOST,1433' -- specified TCP port at specified host
r'SQLHOST:1433' -- the same as above
r'SQLHOST,5000' -- if you have set up an instance to listen on port 5000
r'SQLHOST:5000' -- the same as above

user
database user to connect as.
password
user's password.
trusted
bolean value signalling whether to use Windows Integrated Authentication to connect instead of SQL autentication with user and password [*Windows only*]
charset
character set name to set for the connection.
database
the database you want initially to connect to, by default SQL Server selects the database which is set as default for specific user.
max_conn
how many simultaneous connections to allow; default is 25, maximum on Windows is 1474559; trying to set it to higher value results in error 'Attempt to set maximum number of DBPROCESSes lower than 1.' (error 10073 severity 7) *(added in pymssql 1.0.2)*.

MssqlConnection object properties.

connected
True if the connection object has an open connection to a database, false otherwise.

charset

Character set name that was passed to _mssql.connect() method.

identity
Returns identity value of last inserted row. If previous operation did not involve inserting a row into a table with identity column, None is returned. Example usage -- assume that 'persons' table contains an identity column in addition to 'name' column:

```
conn.execute_non_query("INSERT INTO persons (name) VALUES('John Doe')")
print "Last inserted row has id = " + conn.identity
```

query_timeout
Query timeout in seconds, default is 0, what means to wait indefinitely for results. Due to the way DB-Library for C works, setting this property affects all connections opened from current Python script (or, very technically, all connections made from this instance ofdbinit()).

rows_affected
Number of rows affected by last query. For SELECT statements this value is only meaningful after reading all rows.

debug_queries
If set to true, all queries are printed to stderr after formatting and quoting, just before being sent to SQL Server. It may be helpful if you suspect problems with formatting or quoting.

MssqlConnection object methods.

cancel()
Cancel all pending results from the last SQL operation. It can be called more than one time in a row. No exception is raised in this case.

close()
Close the connection and free all memory used. It can be called more than one time in a row. No exception is raised in this case.

execute_query(query_string)
execute_query(query_string, params)
This method sends a query to the MS SQL Server to which this object instance is connected. An exception is raised on failure. If there are pending results or rows prior to executing this command, they are silently discarded. After calling this method you may iterate over the connection object to get rows returned by the query. You can use Python formatting and all values get properly quoted. Please see examples at the top of this page for details. This method is intented to be used on queries that return results, i.e. SELECT.

execute_non_query(query_string)
execute_non_query(query_string, params)
This method sends a query to the MS SQL Server to which this object instance is connected. After completion, its results (if any) are discarded. An exception is raised on failure. If there are pending results or rows prior to executing this command, they are silently discarded. You can use Python formatting and all values get properly quoted. Please see examples at the top of this page for details. This method is useful for INSERT, UPDATE, DELETE, and for Data Definition Language commands, i.e. when you need to alter your database schema.

execute_scalar(query_string)
execute_scalar(query_string, params)
This method sends a query to the MS SQL Server to which this object instance is connected, then returns first column of first row from result. An exception is raised on failure. If there are pending results or rows prior to executing this command, they are

silently discarded. You can use Python formatting and all values get properly quoted. Please see examples at the top of this page for details. This method is useful if you want just a single value from a query, as in the example below. This method works in the same way asiter(conn).next()[0]. Remaining rows, if any, can still be iterated after calling this method. Example usage:

```
count = conn.execute_scalar("SELECT COUNT(*) FROM employees")
```

execute_row(query_string)
execute_row(query_string, params)

This method sends a query to the MS SQL Server to which this object instance is connected, then returns first row of data from result. An exception is raised on failure. If there are pending results or rows prior to executing this command, they are silently discarded. You can use Python formatting and all values get properly quoted. Please see examples at the top of this page for details. This method is useful if you want just a single row and don't want or don't need to iterate over the connection object. This method works in the same way as iter(conn).next() to obtain single row. Remaining rows, if any, can still be iterated after calling this method. Example usage:

```
empinfo = conn.execute_row("SELECT * FROM employees WHERE empid=10")
```

get_header()

This method is infrastructure and don't need to be called by your code. Get the Python DB-API compliant header information. Returns a list of 7-element tuples describing current result header. Only name and DB-API compliant type is filled, rest of the data is None, as permitted by the specs.

nextresult()

Move to the next result, skipping all pending rows. This method fetches and discards any rows remaining from current operation, then it advances to next result (if any). Returns True value if next set is available, None otherwise. An exception is raised on failure.

select_db(dbname)

This function makes given database the current one. An exception is raised on failure.

__iter__(), next()

These methods faciliate Python iterator protocol. You most likely will not call them directly, but indirectly by using iterators.

_mssql module exceptions

Exception hierarchy.

```
MssqlException
|
+-- MssqlDriverException
|
+-- MssqlDatabaseException
```

MssqlDriverException is raised whenever there is a problem within _mssql -- e.g. insufficient memory for data structures, and so on.

MssqlDatabaseException is raised whenever there is a problem with the database -- e.g. query syntax error, invalid object name and so on. In this case you can use the following properties to access details of the error:

number

The error code, as returned by SQL Server.

severity
The so-called *severity level*, as returned by SQL Server. If value of this property is less than the value of_mssql.min_error_severity, such errors are ignored and exceptions are not raised.

state
The third error code, as returned by SQL Server.

message
The error message, as returned by SQL Server.

You can find an example of how to use this data at the bottom of _mssql examples page.

Frequently asked questions / troubleshooting page

Cannot connect to SQL Server.

If you can't connect to the SQL Server instance, try the following:

- by default SQL Server 2005 and newer doesn't accept remote connections, you have to use SQL Server Surface Area Configuration and/or SQL Server Configuration Manager to enable specific protocols and network adapters; don't forget to restart SQL Server after making these changes,
- if SQL Server is on remote machine, check whether connections are not blocked by any intermediate firewall device, firewall software, antivirus software, or other security facility,
- if you use pymssql on Linux/*nix with FreeTDS, check that FreeTDS's configuration is ok and that it can be found by pymssql. The easiest way is to test connection using tsql utility which can be found in FreeTDS package.
- if you use pymssql on Windows and the server is on local machine, you can try the following command from the command prompt:
  REG ADD HKLM\Software\Microsoft\MSSQLServer\Client /v SharedMemoryOn /t REG_DWORD /d 1 /f

"Unicode data in a Unicode-only collation or ntext data cannot be sent to clients using DB-Library" error appears.

In SQL 2000 SP4 or newer, SQL 2005 or SQL 2008, if you do a query that returns NTEXT type data, you may encounter the following exception:
_mssql.MssqlDatabaseError: SQL Server message 4004, severity 16, state 1, line 1:
Unicode data in a Unicode-only collation or ntext data cannot be sent to clients using DB-Library (such as ISQL) or ODBC version 3.7 or earlier.

It means that SQL Server is unable to send Unicode data to pymssql, because of shortcomings of DB-Library for C. You have to CAST or CONVERT the data to equivalent NVARCHAR data type, which does not exhibit this behaviour.

Column names get silently truncated to 30 characters.

The only workaround is to alias column names to something shorter. Thanks Sebastian Auriol for the suggestion.

```
SELECT very_very_long_column_name_longer_than_30_characters AS col1
```

CHAR(n) and VARCHAR(n) strings get truncated to 255 characters.

This is known limitation of TDS protocol. You can CAST or CONVERT the data to TEXT data type to workaround this issue.

Returned dates are not correct.

If you use pymssql on Linux/*nix and you suspect that returned dates are not correct, please read FreeTDS and dates page.

Shared object "libsybdb.so.3" not found.

On Linux/*nix you may encounter the following behaviour:
```
>>> import _mssql
Traceback (most recent call last):
File "<stdin>", line 1, in ?
ImportError: Shared object "libsybdb.so.3" not found
```

It may mean that FreeTDS library is unavailable, or that dynamic linker is unable to

find it. Check that it is installed and that the path to libsybdb.so is in /etc/ld.so.conf file. Then do ldconfig as root to refresh linker database. On Solaris I just set LD_LIBRARY_PATH environment variable to directory with the library just before launching Python.

"*DB-Lib error message 20004, severity 9: Read from SQL server failed*" *error appears.*

On Linux/*nix you may encounter the following behaviour:

```
>>> import _mssql
>>> c=_mssql.connect('hostname:portnumber','user','pass')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
_mssql.DatabaseException: DB-Lib error message 20004, severity 9:
Read from SQL server failed.
DB-Lib error message 20014, severity 9:
Login incorrect.
```

It may happen when one of the following is true:

- freetds.conf file cannot be found,
- tds version in freetds.conf file is not 7.0 or 4.2,
- any character set is specified in freetds.conf,
- an unrecognized character set is passed to _mssql.connect() or pymssql.connect() method.

"Login incorrect" following this error is spurious, real "Login incorrect" messages has code=18456 and severity=14.

Python on Windows dies with memory access violation error on connect() when incorrect password is given.

This may happen if you use different version of ntwdblib.dll than the one included in pymssql package. For example the version 2000.80.2273.0 is unable to handle dberrhandle() callbacks properly, and causes access violation error in err_handler() function on return INT_CANCEL. I have given up after several hours of investigating the issue, and just reverted to previous version of the ntwdblib.dll and error disappeared.

"*Not enough storage is available to complete this operation*" *error appears.*

On Windows you may encounter the following behaviour:

```
>>> import _mssql
>>> c=_mssql.connect('hostname:portnumber','user','pass')
Traceback (most recent call last):
File "<pyshell#1>", line 1, in -toplevel-
File "E:\Python24\Lib\site-packages\pymssql.py", line 310, in connect
con = _mssql.connect(dbhost, dbuser, dbpasswd)
error: DB-Lib error message 10004, severity 9:
Unable to connect: SQL Server is unavailable or does not exist. Invalid connection.
Net-Lib error during ConnectionOpen (ParseConnectParams()).
Error 14 - Not enough storage is available to complete this operation.
```

This may happen most likely on earlier versions of pymssql. It happens always if you use a colon ":" to separate hostname from port number. On Windows you should use comma "," instead. pymssql 1.0 has a workaround, so you do not have to care about that.

More troubleshooting.

If the above hasn't covered the problem, please also check Limitations and known issues page. You can also consult FreeTDS troubleshooting page for issues related to the TDS protocol.

Limitations and known issues

pymssql does not support an 'elegant' way of handling stored procedures. Nonetheless you can fully utilize stored procedures, pass values to them, fetch rows and output parameter values. See _mssql examples.

DB-Library for C is not supported by Microsoft any more. You can find the info on this MSDN page. This is why some of the features may not work as expected. Here are some known issues and workarounds. You should note that *none of these issues are imposed by pymssql*. You will observe them all also in PHP driver for MSSQL, for instance.

- **image data is truncated to 4000 characters.**
  This is known limitation of DB-Library for C. I know of no workaround. This issue is also present in PHP, the solution suggested was to use ODBC protocol.

- **varchar and nvarchar data is limited to 255 characters, and longer strings are silently trimmed.**
  This is known limitation of TDS protocol. A workaround is to CAST or CONVERT that row or expression to text data type, which is capable of returning 4000 characters.

- **column names are limited to 30 characters and longer names are silently truncated.**
  There's no workaround for this. You have to use names (or aliases as in SELECT column AS alias) that are not longer than 30 characters.

- **"SELECT '" statement returns a string containing one space instead of an empty string.**
  There's no workaround for this. You cannot distinguish between
  ```
  SELECT ''   -- empty string
  ```
  and
  ```
  SELECT ' '  -- one space
  ```

- **"SELECT CAST(NULL AS BIT)" returns False instead of None.**
  There's no workaround for this. You cannot distinguish between
  ```
  SELECT CAST(NULL AS BIT)
  ```
  and
  ```
  SELECT CAST(0 AS BIT)
  ```

  You should avoid NULL bit fields. Just assign a default value to them and update all records to the default value. I would recommend not using BITdatatype at all, if possible change it to TINYINT for example. The problem will disappear, and storage overhead is unnoticeable. This issue is also known for example in Microsoft's own product, Access, see KB278696 article.

- **New features of SQL Server 2005 and SQL Server 2008 are not supported.**
  Some of the features of SQL Server versions newer than 2000 just don't work. For example you can't use MARS feature (Multiple Active Result Sets). You have to arrange your queries so that result sets are fetched one after another.

- **The newest version of ntwdblib.dll library v. 2000.80.2273.0 is unusable.**
  If on Windows, please use the library bundled with pymssql package. Older or newer versions may introduce unexpected problems, the version mentioned

above causes memory violation errors, but only in certain scenarios, making tracing the cause very difficult. More information is also available on FAQ page.

FreeTDS and dates

Summary: make sure that FreeTDS is compiled with --enable-msdblib configure option, or your queries will return wrong dates -- 2009-00-01 instead of 2009-01-01.

There's an obscure problem on Linux/*nix that results in dates shifted back by 1 month. This behaviour is caused by different dbdatecrack() prototypes in Sybase Open Client DB-Library/C and the Microsoft SQL DB Library for C. The first one returns month as 0..11 whereas the second gives month as 1..12. See this FreeTDS mailing list post, Microsoft manual for dbdatecrack(), and Sybase manual for dbdatecrack() for details.

FreeTDS, which is used on Linux/*nix to connect to Sybase and MS SQL servers, tries to imitate both modes:

- default behaviour, when compiled *without* --enable-msdblib, gives dbdatecrack() which is Sybase-compatible,
- when configured with --enable-msdblib, the dbdatecrack() function is compatible with MS SQL specs.

pymssql requires MS SQL mode, evidently. Unfortunately at runtime we can't reliably detect which mode FreeTDS was compiled in (as of FreeTDS 0.63). Thus at runtime it may turn out that dates are not correct. If there was a way to detect the setting, pymssql would be able to correct dates on the fly.

If you can do nothing about FreeTDS, there's a workaround. You can redesign your queries to return string instead of bare date:

```
SELECT datecolumn FROM tablename
```

can be rewritten into:

```
SELECT CONVERT(CHAR(10),datecolumn,120) AS datecolumn FROM tablename
```

This way SQL will send you string representing the date instead of binary date in datetime or smalldatetime format, which has to be processed by FreeTDS and pymssql.

On Windows there's no problem at all, because we link with MS library, which is compatible with SQL Server, obviously.

Advanced information

Connections: hosts, protocols and aliases.

This information covers Windows clients. I documented it here because I found no other place where these details are described. The same advanced options are available for Linux/*nix clients using FreeTDS library, you can set up host aliases in freetds.conf file. Look for information in the FreeTDS documentation.

If you need to connect to a host using specified protocol, e.g. named pipes, you can set up a specific aliased connection for the host using Client Network Utility, which is bundled on SQL Server 2000 installation media. If you don't have one, you can do the same by creating Registry entries. Here are some examples of how to proceed by making changes to the Registry.

These entries in fact create *aliases* for host names.

Example 1. Connect to host sqlhost3 with named pipes.
Execute the following command from the command line (this is one line, don't break it):

```
REG ADD
"HKLM\Software\Microsoft\MSSQLServer\Client\ConnectTo
" /v sqlhost3 /t REG_SZ /d
"DBNMPNTW,\\sqlhost3\pipe\sql\query"
```

then from pymssql connect as usual, giving just the string 'sqlhost3' as host parameter to connect() method. This way you only provide the alias sqlhost3 and the driver looks for the settings in Registry. The above path \\sqlhost3\pipe\sql\query usually means default SQL Server instance on sqlhost3 machine. You have to consult your configuration (either with Server Network Utility or SQL Server Configuration Manager) to obtain path for a named instance.

Example 2. Connect to host sqlhost4 with TCP/IP protocol.
It may seem strange at first, but there are chances that the client machine's Registry is set up so preferred protocol is named pipes, and one may want to set an alias for a specific machine manually. Execute the following command from the command line (this is one line, don't break it):

```
REG ADD
"HKLM\Software\Microsoft\MSSQLServer\Client\ConnectTo
" /v sqlhost4 /t REG_SZ /d
"DBMSSOCN,sqlhost4.example.com,1433"
```

then from pymssql connect as usual, giving just the string 'sqlhost4' as host parameter to connect() method. This way you only provide the alias sqlhost4 and the driver looks for the settings in Registry. As you can see, there is host name and TCP port number hard coded for the alias sqlhost4.

Compilation and installation from source

If you need to compile pymssql, check whether requirements shown below are met, unpack source files to a directory of your choice and issue (as root):
python setup.py install
This will compile and install pymssql.

Build Requirements

- Python language. Please check platforms page for version info.
- Python development package -- when needed by your OS (for example python-dev or libpython2.5-devel).
- Linux, *nix and Mac OS X: FreeTDS 0.63 or newer (you need freetds-dev or freetds-devel or similar named package -- thanks Scott Barr for pointing that out).
  NOTE: FreeTDS must be configured with --enable-msdblib to return correct dates! See FreeTDS and Dates for details.
- Windows: SQL Developer Tools, to be found on MS SQL 2000 installation media.
- Windows: If you compile pymssql for Python 2.4 or 2.5, you need either Microsoft Visual C++ .NET 2003 or Microsoft Visual Studio .NET 2003. It will complain if you will try to use newer versions.
- Windows: If you compile pymssql for Python 2.6 or newer, you need Microsoft Visual Studio 2005 or Microsoft Visual Studio 2008. I think that downloadable Microsoft Visual C++ Express edition can be used.

Platform-specific issues

Mandriva Linux

If you use some older versions of Mandriva Linux and want to compile pymssql, you may have to edit setup.py and change:

```
    libraries = ["sybdb"]
```

into:

```
    libraries = ["sybdb_mssql"]
```

Be sure to install libfreetds_mssql0 package first.

Windows

FreeTDS on Windows is not supported.

Please also consult FreeTDS and Dates document.